

Salvemos Pythonia

Atención, programadores iniciados:

El reino de Pythonia está en apuros. Los monstruos están aburridos porque nadie los programa, las mazmorras están vacías porque nadie las visita, y los héroes... bueno... no existen porque NADIE LOS HA CODIFICADO.

¡Así no se puede vivir!

Tu misión es clara:

Debes crear tu propio juego de rol antes de que los monstruos empiecen a hacer huelga...

Descripción del proyecto

En este proyecto vas a desarrollar un mini juego de rol por consola utilizando Programación Orientada a Objetos en Python.

El jugador:

- Creará un grupo de 3 héroes eligiendo entre varias clases (Guerrero, Mago, Asesino, Clérigo).
- Se enfrentará a un grupo fijo de enemigos (goblins y orcos).
- Resolverá el combate mediante un sistema de turnos, ordenado por la velocidad de cada combatiente.
- Podrá elegir en cada turno entre ataque básico, habilidad especial o pasar turno.

El objetivo es aplicar POO de forma práctica (clases, herencia, métodos, atributos), trabajar con funciones y gestionar el flujo de un programa interactivo.

Requisitos del proyecto

- El proyecto debe estar desarrollado en Python 3.
- El código debe estar organizado en clases y funciones, no todo en el bloque principal.
- Se debe utilizar entrada por teclado (input) para las decisiones del jugador.
- Se valorará:
 - Claridad y organización del código.
 - Uso adecuado de POO.
 - Manejo de errores básicos en la entrada del usuario.

PARTE 1 – Modelo de personajes (héroes)

1.1. Clase base Personaje

Debes crear una clase Personaje que represente a cualquier héroe del juego.

Atributos mínimos:

- **nombre (string):** nombre del personaje.
- **clase (string):** tipo de personaje ("Guerrero", "Mago", etc.).
- **vida (int):** puntos de vida máximos.
- **ataque_base (int):** daño base del personaje (dado de ataque)
- **clase_armadura (int):** dificultad para impactarle (CA).
- **velocidad (int):** determina el orden de actuación en el combate.

Métodos mínimos:

- **esta_vivo(self):** Devuelve True si la vida actual del personaje es mayor a 0
- **recibir_daño(self, cantidad):** Resta la cantidad de vida al personaje (sin bajar de 0) e informa por pantalla del daño recibido y su vida actual.
- **tirar_d20(self):** Devuelve un número aleatorio entero entre 1 y 20 (simulación de un dado de 20 caras).
- **atacar(self, objetivo, atributo, dado_ataque):** Implementa un ataque genérico:
 - Realiza una tirada d20.
 - Suma el modificador de atributo (por ejemplo fuerza, destreza, etc.)(estos atributos se crearán más adelante en la creación de clases).
 - Si el resultado es mayor o igual que la CA del objetivo, el ataque impacta y se aplica un daño aleatorio entre 1 y el dado de ataque de cada clase.
 - En caso contrario, el ataque falla.
- **__str__(self):** Devuelve un texto legible con la información básica del personaje (nombre, clase, vida, CA, velocidad).

1.2. Subclases de héroes

Debes crear las siguientes subclases que hereden de Personaje:

- **Guerrero(Personaje)**
- **Mago(Personaje)**
- **Asesino(Personaje)**
- **Clerigo(Personaje)**

Cada subclase:

- Llamará al constructor de Personaje mediante **super().__init__(...)** con sus propios valores. Debes definir los valores que creas justos para cada clase.
- Tendrá al menos un atributo propio (por ejemplo: **fuerza, inteligencia, destreza, fe**).
- Implementará un método **atacar_basico(self, objetivo)** que llame internamente al método genérico **atacar(...)** de la clase base indicando el atributo y el dado de ataque (**ataque_base**) a utilizar.

Además, cada clase tendrá una habilidad especial:

- **Guerrero.abocajarro(self, objetivo)**
 - Tirada d20 + fuerza.
 - Si impacta, hace un daño mayor (por ejemplo, un valor aleatorio entre 1 y **ataque_base + fuerza**).
 - Como penalización, el propio guerrero recibe parte del daño (por ejemplo, **fuerza/2**).
- **Mago.bola_fuego(self, objetivo)**
 - Tirada d20 + inteligencia.
 - Si impacta, hace un daño basado en la inteligencia (por ejemplo, random entre 1 e **inteligencia * 2**).
- **Asesino.ataque_critico(self, objetivo)**
 - Tirada d20 + destreza.
 - Si impacta:
 - Si la tirada de d20 es par → daño crítico (por ejemplo, el doble del random entre 1 y **daño base**).

- Si es impar → daño reducido (por ejemplo, la mitad del random entre 1 y daño base).
- **Clerigo.curar(self, objetivo)**
 - No es un ataque, sino una habilidad de apoyo.
 - Cura a un aliado una cantidad de vida basada en su atributo de fe (por ejemplo, random entre 1 y fe * 2), sin superar la vida máxima del objetivo.

PARTE 2 – Modelo de monstruos

2.1. Clase base Monstruo

Crea una clase Monstruo que representará a los enemigos (es muy similar a la clase Personaje).

Atributos mínimos: nombre, ataque, vida, clase de armadura, velocidad

Métodos mínimos: son similares a los de la clase personaje

- **esta_vivo(self)**
- **recibir_daño(self, cantidad)**
- **tirar_d20(self)**
- **atacar(self, heroe)**
 - Tirada d20 + ataque.
 - Si el total es mayor o igual que la CA del héroe, el ataque impacta y se causa un daño (un valor aleatorio entre 1 y el ataque base del monstruo).
- **__str__(self)**
Muestra por pantalla la información básica del monstruo.

2.2. Tipos concretos de monstruos

Crea al menos dos subclases:

- **Goblin(Monstruo)**
- **Orco(Monstruo)**

Cada una define en su `__init__` sus propios valores para ataque, vida, CA y velocidad.

PARTE 3 – Creación del grupo de héroes (menú de campaña)

Implementa un **menú de creación de personajes** que permita al jugador formar su grupo de héroes antes de comenzar la aventura.

3.1. Funciones requeridas

- **elegir_clase():** Muestra las clases disponibles (Guerrero, Mago, Asesino, Clérigo) y devuelve la clase elegida.
- **crear_heroe():**
 - Pide al usuario el nombre del héroe.
 - Llama a `elegir_clase()`.
 - Crea un objeto del tipo correspondiente.
 - Devuelve el héroe creado.
- **mostrar_grupo(grupo):** Muestra por pantalla todos los héroes del grupo.
- **eliminar_heroe(grupo):** Permite al usuario elegir qué héroe del grupo eliminar.
- **menu_creacion_campaña():** muestra un menú similar al siguiente:

```
=====
      CREACIÓN DE PERSONAJES DE CAMPAÑA
=====
Héroes en el grupo: 0/3
1. Añadir nuevo héroe
2. Ver grupo actual
3. Eliminar héroe
4. Terminar creación y continuar
=====
```

- Debe permitir:
 - Añadir héroes hasta un máximo de 3.
 - Ver el grupo en cualquier momento.
 - Eliminar uno de los héroes creados.
 - Finalizar solo si el grupo tiene exactamente 3 héroes.

La función debe devolver una lista con el grupo de héroes cuando termine.

PARTE 4 – Sistema de combate por turnos

4.1. Combate

Crea una función **combate(grupo_heroes, grupo_monstruos)** que:

- Inicie un contador de rondas.
- Mientras haya héroes vivos y monstruos vivos:
 - Muestre el número de ronda.
 - Cree una lista de combatientes (héroes vivos + monstruos vivos).
 - Ordene esta lista por velocidad (mayor primero). A igualdad de velocidad, los héroes actúan antes que los monstruos.
 - Recorra la lista ordenada:
 - Si es un héroe → llama a la función turno_herroe.
 - Si es un monstruo → llama a la función turno_monstruo.
- Cuando termine el combate:
 - Si quedan héroes vivos → muestra mensaje de victoria.
 - Si no → muestra mensaje de derrota.

4.2. Gestión de vivos

Implementa dos funciones de ayuda:

- **heroes_vivos(grupo_heroes)**: Devuelve una lista con los héroes cuya vida sea > 0 .
- **monstruos_vivos(grupo_monstruos)**: Devuelve una lista con los monstruos cuya vida sea > 0 .

4.3. Elección de objetivos

- **elegir_objetivo_monstruo(monstruos)**: Muestra los monstruos vivos y permite seleccionar por número cuál atacar.
- **elegir_aliado_para_curar(grupo_heroes)**: Muestra los héroes vivos y permite elegir a cuál curar (para el Clérigo).

4.4. Turno de héroe

Implementa **turno_heroe(heroe, grupo_heroes, grupo_monstruos)**:

- Si el héroe está vivo:
 - Muestra su nombre, clase y vida actual.
 - Ofrece un pequeño menú con tres acciones:
 - 1. Atacar (básico): elegir monstruo y usar atacar_basico.
 - 2. Usar habilidad:
 - Clérigo: elegir aliado y usar curar.
 - Guerrero/Mago/Asesino: elegir monstruo y usar su habilidad especial correspondiente.
 - 3. Pasar turno: no hace nada ese turno.

4.5. Turno de monstruo

Implementa **turno_monstruo(monstruo, grupo_heroes)**:

- Si el monstruo está vivo:
 - Elige aleatoriamente un héroe vivo.
 - Llama al método atacar(...) del monstruo sobre ese héroe.

PARTE 5 – Aventura

Crea una función **aventura(grupo_heroes)** que:

1. Muestre un mensaje de inicio de aventura.
2. Genere un encuentro fijo de enemigos, por ejemplo:
 - 2 goblins y 1 orco:
 - grupo_monstruos = [Goblin(), Goblin(), Orco()]
3. Muestre por pantalla la lista de enemigos (nombre, vida, CA...).
4. Llame a combate(grupo_heroes, grupo_monstruos).
5. Muestre un mensaje final indicando si el grupo ha sobrevivido o no.

En el bloque principal (if __name__ == "__main__":):

- Llama primero a menu_creacion_campaña() para obtener el grupo de héroes.
- Después, llama a aventura(grupo_heroes).

Ampliaciones opcionales:

No forman parte del mínimo obligatorio, pero podrán valorarse como mejora:

- Nuevos tipos de héroes o monstruos.
- Nuevas habilidades especiales con efectos distintos.
- Más de un combate (varias oleadas).
- Sistema de experiencia y subida de nivel.
- Sistema de objetos o inventario (pociones de curación, por ejemplo).